

Article title

Open-Source Closed-Loop DC Motor Driver with Embedded PID Control and Trapezoidal Trajectory Planning

Authors

Author details removed for double-blind review

Affiliations

Affiliation removed for double-blind review

Corresponding author's email address and Twitter handle

ail removed for double-blind review

Abstract

An open-source, closed-loop DC motor driver is presented for motion control in robotics and automation applications. The hardware integrates a full H-bridge, a magnetic or optical encoder interface, and a high-frequency PID feedback controller on a compact PCB centred on the STM32C0 microcontroller. An embedded trapezoidal trajectory planner generates smooth motion profiles by managing acceleration, cruise, and deceleration phases, eliminating host-side trajectory computation. The driver supports a 5–45 V supply range per the DRV8251A specification; all validation was conducted at 12 V. It delivers up to 4.1 A peak current; continuous operation above approximately 1.6 A RMS requires enlarged copper pours or active cooling. Communication is supported via I2C and UART using a simple ASCII protocol, enabling integration with any microcontroller or single-board computer. Up to 112 units can be daisy-chained on a single I2C bus. All design files, firmware, schematics, and Gerber files are released under the MIT License and are publicly available. Closed-loop position control and trapezoidal trajectory tracking were validated experimentally, demonstrating zero overshoot under proportional-only control and cycle-to-cycle repeatability of 1.6 encoder counts (0.67°) across five consecutive trajectories.

Keywords

closed-loop motor control; DC motor driver; embedded PID; open-source hardware; STM32; trapezoidal trajectory

Specifications table

Hardware name	Closed-Loop DC Motor Controller
Subject area	Engineering and material science
Hardware type	Electrical engineering and computer science
Closest commercial analog	ODrive Motor Controller; Pololu Jrk G2 Motor Controller; Dynamixel Smart Servo Actuator. Unlike these, the present design is fully open-source, operates over a wide voltage range (5–45 V, rated per component specification; validated at 12 V), and supports native I2C daisy-chaining without proprietary software.
Open source license	MIT License
Cost of hardware	Approximately \$3.24 per unit at prototype scale (PCB in batches of 10 from JLCPCB; components from DigiKey/LCSC at single-unit pricing), representing a 50–100× reduction compared to commercial equivalents such as ODrive (\$200–\$500) and Dynamixel (\$50–\$400 per axis). A one-time SMT stencil cost of \$7.00 applies.
Source file repository	https://doi.org/10.5281/zenodo.20087909

1. Hardware in context

Closed-loop motor control is a fundamental requirement in robotics, automated manufacturing, rehabilitation devices, and precision instrumentation. A closed-loop driver continuously monitors motor position or velocity through an encoder and adjusts the drive signal in real time to minimise the error between the commanded and actual state. While open-loop stepper motors remain common in cost-sensitive designs, they are prone to stall and position loss under varying load conditions. Closed-loop DC motor systems offer superior dynamic performance and energy efficiency.

Several commercial and open-source solutions address this problem. The **ODrive S1** controller is a high-performance open-source platform that supports field-oriented control for brushless motors, but is priced in the \$200–\$500 range, targets brushless motor configurations, and is designed for a different application tier. The **Pololu Jrk G2** series offers closed-loop DC motor control at a lower cost but lacks an onboard trajectory planner and supports only one motor per board. **Dynamixel** servo actuators integrate all electronics into the actuator itself, making them expensive (\$50–\$400+ per axis) and incompatible with third-party motors. Other commercial motor driver ICs (e.g., DRV8833, TB6612FNG) provide H-bridge drive but no onboard intelligence, requiring all control logic to be implemented externally.

The hardware presented here occupies a distinct position: a self-contained motor driver that accepts position or velocity commands and handles all low-level control onboard. Open-source hardware platforms have demonstrated significant value for reproducible research and rapid prototyping [11,12]. This design achieves a 50–100× cost reduction relative to commercial equivalents while remaining compact, fully open-source, and scalable through I2C daisy-chaining. The STM32C0 microcontroller runs the PID loop at sufficient speed for the target applications, and the onboard trajectory generator removes the need for host-side motion planning. The ASCII-based command interface works with any host platform capable of serial communication.

Beyond general robotics, the driver is applicable to several scientific instrument contexts. In motorised positioning stages, the closed-loop encoder feedback and trajectory planner provide repeatable automated sample placement without requiring a dedicated motion controller. In rehabilitation engineering research, the accessible price point and I2C daisy-chaining allow multi-joint actuator arrays to be assembled and controlled from a single microcontroller, reducing system complexity in prototype exoskeleton and assistive device studies. In laboratory automation, the driver can actuate pipetting stages, filter wheels, or valve actuators in custom instruments where commercial smart actuators are prohibitively expensive. The open-source firmware also makes the platform directly suitable as a teaching instrument in control systems and mechatronics courses, where students can modify and extend the PID and trajectory code without proprietary toolchains.

2. Hardware description

The Closed-Loop DC Motor Controller is a single-board closed-loop motor controller for brushed DC motors equipped with incremental magnetic or optical encoders. The board integrates the drive stage, microcontroller, and feedback controller into a compact form factor (30 mm × 40 mm), suitable for multi-axis robotic systems.

At the core of the design is the **STM32C011F6U6TR** microcontroller, running at 48 MHz (HSI oscillator, no division), which executes a discrete PID feedback loop [8,9] that computes motor drive commands based on the error between the desired and measured position. The 48 MHz clock provides a high PID loop update rate; static analysis of the main loop estimates 100–200 kHz under idle conditions, with the rate reduced during active I2C or UART communication. An **H-bridge** drive stage (DRV8251ADDAR) supports bidirectional motor control with a supply voltage range of 5 V to 45 V (rated per component specification; validated at 12 V) and continuous current up to 4.1 A. The H-bridge is driven at approximately 11.7 kHz PWM, eliminating audible motor buzz and reducing current ripple. The encoder interface uses TIM3 in hardware quadrature mode with a 64-bit software counter extension, allowing virtually unlimited position range without rollover. Motor current is measured via a 12-bit ADC reading the IPROPI output of the DRV8251A; quantitative torque estimation is a planned capability pending hardware filtering (see Section 7.4). The board is shown assembled in Figure 1.

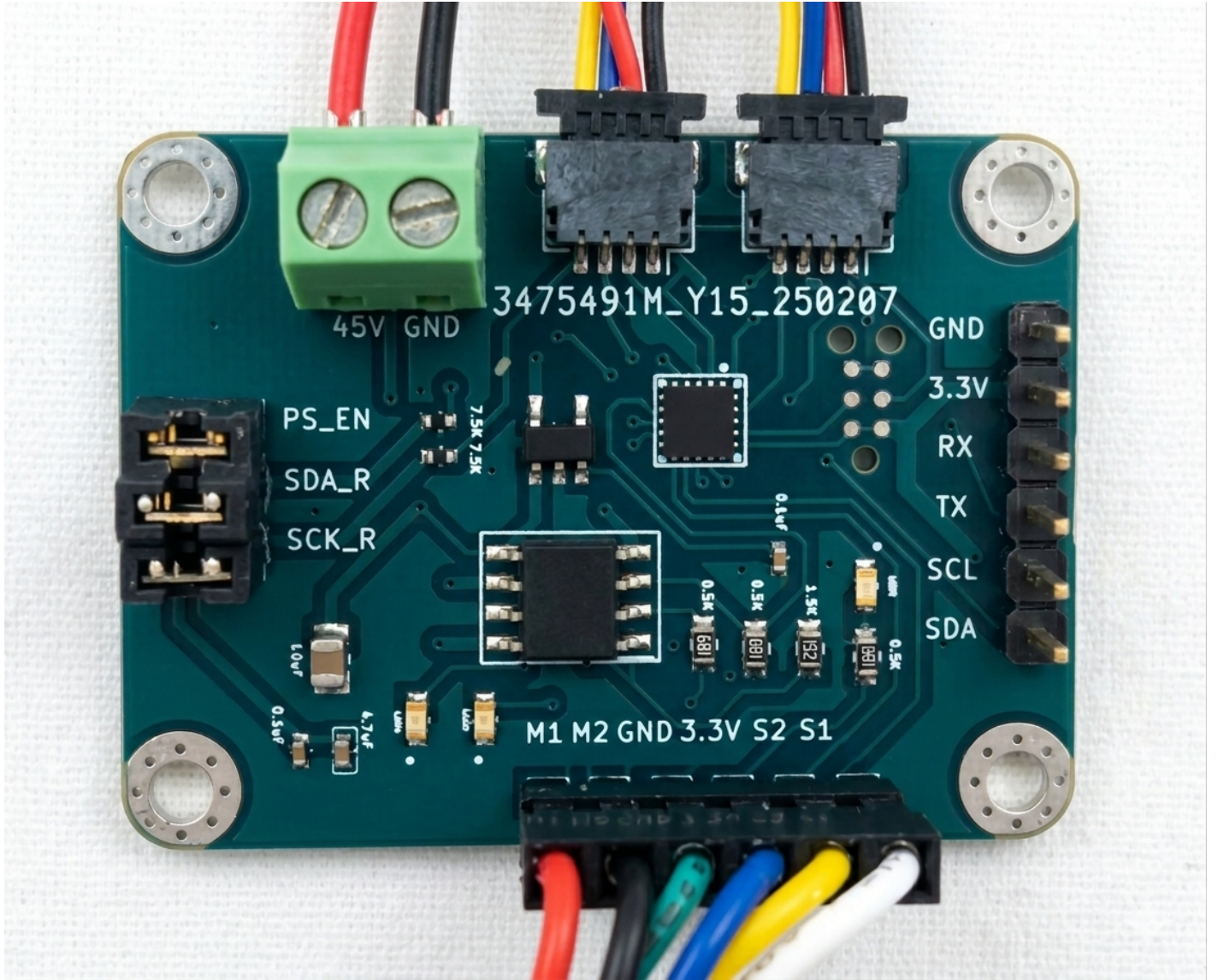


Figure 1: Fully assembled Closed-Loop DC Motor Controller PCB (30 mm $\times$ 40 mm).

A key feature is the **embedded trapezoidal trajectory planner** [10]. Rather than requiring the host to stream continuous setpoints, a target position is specified once; the controller internally computes and executes a smooth motion profile respecting configurable limits on velocity, acceleration, and deceleration. This eliminates mechanical shock, reduces encoder noise sensitivity during rapid movements, and simplifies host software.

Communication is available over **UART** (via a USB-to-UART adapter for PC connection, or directly to any microcontroller) and **I2C** via **Qwiic-compatible connectors** for easy daisy-chaining. The ASCII command protocol supports setting target positions, reading back current position and velocity, querying motor current, and tuning PID gains at runtime without reflashing firmware. Up to 112 units can share a single I2C bus using standard 7-bit addressing.

The key utilities of this hardware for the research community are summarised below:

- **Multi-axis robotics:** Up to 112 drivers can be daisy-chained on a single I2C bus, making the platform well-suited for robotic arms, hexapods, and other systems requiring coordinated multi-joint control.
- **Rapid prototyping:** The ASCII command interface and Qwiic connectors enable integration into experimental setups without custom firmware or dedicated driver libraries.
- **Educational platforms:** The open-source firmware and simple communication protocol make this an effective teaching tool for control systems, embedded systems, and mechatronics courses.
- **Accessible automation:** At approximately \$3.24 USD per unit — a 50–100 $\times$ reduction compared to

commercial smart actuator systems — the driver enables laboratory automation, sample handling, and multi-axis positioning in research settings where commercial solutions are cost-prohibitive.

- **Research instrumentation:** The hardware includes an onboard motor current sense circuit for qualitative load monitoring. Quantitative current sensing is a planned capability for a future hardware revision; it is not validated in the present work.

Design files

All design files are released under the MIT License and are publicly available in the Zenodo repository:

<https://doi.org/10.5281/zenodo.20087909>

The repository contains PCB design files in IPC-2581C, CAD, and Flux formats, plus schematic as PNG and Gerber files for fabrication; complete STM32CubeIDE firmware source in C with extensive inline comments and a pre-compiled binary for direct flashing; and PCB render images together with a full assembly walkthrough [video](#).

Note on editability: IPC-2581C import is not yet supported in KiCad; users working in a FOSS EDA environment are recommended to use the Gerber files for fabrication and the schematic PNG as a reference. A native KiCad source export is planned for a future repository update.

3. Design files summary

Design filename	File type	Open source license	Location of the file
PCB_Design/schematic.png	Schematic image (PNG)	MIT License	Zenodo repository
PCB_Design/schematic.cad	PCB design (CAD)	MIT License	Zenodo repository
PCB_Design/schematic.ipc2581c	PCB design (IPC-2581C)	MIT License	Zenodo repository
PCB_Design/schematic.flx	PCB design (Flux)	MIT License	Zenodo repository
PCB_Design/schematic.zip	PCB design archive (all formats)	MIT License	Zenodo repository
gerber_files/	Gerber and drill files (fabrication)	MIT License	Zenodo repository
SourceCode/Src/main_original.c	Firmware source (C)	MIT License	Zenodo repository
SourceCode/Inc/helper.h	Firmware header (C)	MIT License	Zenodo repository
firmware_Image.elf	Pre-compiled firmware binary	MIT License	Zenodo repository
media/pcb_.png	PCB layout render (PNG)	MIT License	Zenodo repository
docs/USER_GUIDE.md	User guide (Markdown)	MIT License	Zenodo repository

File descriptions:

PCB_Design/schematic.png — Full circuit schematic of the motor driver board, showing the STM32C0 microcontroller, H-bridge stage, encoder input conditioning, I2C/UART interfaces, and power supply connections.

PCB_Design/schematic.cad, schematic.ipc2581c, schematic.flx, schematic.zip — PCB design files in CAD, IPC-2581C, and Flux formats, plus a combined archive. IPC-2581C is a vendor-neutral open standard supported by Altium and Cadence; note that KiCad v8 supports IPC-2581 export but not yet import. Users working in a FOSS EDA environment should use the Gerber files for fabrication and the schematic PNG as a reference.

gerber_files/ — Complete set of Gerber and drill files for PCB fabrication, including front/back copper, mask, paste, silkscreen, edge cuts, drill file, netlist (.d356), and pick-and-place CSV. Ready to submit to any PCB manufacturer (e.g., JLCPCB, PCBWay).

SourceCode/Src/main_original.c — Complete embedded firmware in C, implementing the PID controller, trapezoidal trajectory generator, quadrature encoder interface, and UART/I2C ASCII command handler. All major functions are commented.

SourceCode/Inc/helper.h — Header file containing helper function declarations and definitions used by the firmware.

firmware_Image.elf — Pre-compiled firmware binary for direct flashing via STM32CubeProgrammer, without requiring a build toolchain.

media/pcb.png — Rendered top-view image of the final PCB layout for visual reference during assembly.

docs/USER.GUIDE.md — User guide covering command syntax, wiring, and basic operation.

Bill of materials

A complete editable spreadsheet version of the BOM is available in the project repository alongside the design files at <https://doi.org/10.5281/zenodo.20087909>. All pricing is based on single-unit quantities; costs decrease significantly at higher quantities. Components are available through DigiKey, Mouser, or LCSC using the manufacturer part numbers listed.

4. Bill of materials summary

Designator	Component	Number	Cost per unit (USD)	Total cost (USD)	Source of materials	Material type
U1	STM32C011F6U6TR — STMicroelectronics ARM Cortex-M0+ MCU, UFQFPN-20	1	\$0.45	\$0.45	DigiKey , LCSC	Semiconductor
U3	DRV8251ADDAR — Texas Instruments brushed DC motor H-bridge driver, WSON-8	1	\$0.44	\$0.44	DigiKey , LCSC	Semiconductor
IC1	AP7375-33W5-7 — Diodes Inc. 3.3 V LDO voltage regulator, 300 mA, SOT-25	1	\$0.22	\$0.22	DigiKey , LCSC	Semiconductor
J2, J5	SM04B-SRSS-TB — JST SH 4-pin SMD connector, 1.0 mm pitch (Qwiic / I2C)	2	\$0.14	\$0.28	DigiKey	Polymer
J1	KF124-3.5-2P-1G — 2-pin PCB screw terminal, 3.5 mm pitch	1	\$0.15	\$0.15	DigiKey	Polymer
J3	5-pin male header, 2.54 mm pitch	1	\$0.05	\$0.05	DigiKey	Polymer
J4	6-pin male header, 2.54 mm pitch	1	\$0.08	\$0.08	DigiKey	Polymer
LED3, LED5, LED6	LTST-C190TBKT — Lite-On blue SMD LED, 0402	3	\$0.05	\$0.15	DigiKey	Semiconductor
C2	GRM155R61A475MEAAD — Murata 4.7 μ F ceramic capacitor, 0402	1	\$0.02	\$0.02	DigiKey	Ceramic
C3	CL21B106KOQNNNE — Samsung 10 μ F ceramic capacitor, 0805	1	\$0.03	\$0.03	DigiKey	Ceramic
C4, C7	GCM155R71H104KE02J — Murata 100 nF ceramic capacitor, 0402	2	\$0.01	\$0.02	DigiKey	Ceramic
R1	RC0201FR-071R5L — Yageo 1.5 Ω resistor, 0201	1	\$0.01	\$0.01	DigiKey	Non-specific
R2, R6, R7	RC0603JR-10680RL — Yageo 680 Ω resistor, 0603 (LED current limiting)	3	\$0.01	\$0.03	DigiKey	Non-specific
R20, R21	CRCW040210K0FKEDC — Vishay Dale 10 k Ω resistor, 0402 (I2C pull-ups)	2	\$0.01	\$0.02	DigiKey	Non-specific
PS_EN, SCK_PU, SDA_PU	QPC02SXGN-RC — Sullins 2-pin jumper, 2.54 mm pitch	3	\$0.05	\$0.15	DigiKey	Polymer
U6	TC2030 — Tag-Connect 6-pin SWD programming pad (footprint only)	1	\$0.00	\$0.00	PCB footprint only	Non-specific
PCB	Custom 2-layer PCB, 30 mm $\times$ 40 mm (batches of 10)	1	\$0.72	\$0.72	JLCPCB	Non-specific
	Total per unit			\$3.24		

Note: An SMT stencil (\$7.00 one-time cost from JLCPCB) is recommended for paste application. Shipping costs vary by region.

5. Build instructions

5.1 Prerequisites and tools

The following tools and consumables are required to assemble the board:

- Reflow hotplate or oven for SMD component soldering
- Solder paste (lead-free SMD solder paste, e.g., SAC305)
- SMT stencil (recommended for uniform paste application; available from JLCPCB at low cost)
- Flux and isopropyl alcohol (IPC cleaning)
- Fine-tip tweezers for SMD component placement
- Magnification aid (loupe or stereo microscope recommended for 0201 passive components)
- Temperature-controlled soldering iron with a fine conical tip (0.2–0.4 mm) for through-hole connectors
- Multimeter for continuity checks before powering
- ST-Link V2 programmer or compatible SWD debugger for firmware flashing

Safety notice: The board operates at voltages up to 45 V and currents up to 4.1 A. Always verify supply polarity before connection. Never exceed the rated voltage or current. Handle all power connections with care when the board is energized.

5.2 PCB fabrication

Submit the Gerber files located in the `gerber_files/` directory to a PCB fabrication service. Recommended fabrication parameters:

- Layers: 2
- Board dimensions: 30 mm × 40 mm
- Base material: FR-4
- Board thickness: 1.6 mm
- Outer copper weight: 1 oz (35 µm)
- Surface finish: HASL (with lead); lead-free HASL or ENIG acceptable for RoHS-compliant fabrication
- Solder mask: both sides (green)
- Silkscreen: white
- Electrical test: flying probe, fully tested
- Board outline tolerance: ±0.2 mm

5.3 Component assembly

A complete assembly walkthrough is provided in the [assembly video](#). The following steps summarise the soldering sequence:

1. Apply solder paste to all SMD pads using the SMT stencil.
2. Place all SMD components (resistors, capacitors, ICs) using fine-tip tweezers.
3. Reflow the board on a hotplate or in a reflow oven following the solder paste manufacturer’s temperature profile.
4. Inspect all joints under magnification, paying particular attention to U3 (H-bridge) and U1 (STM32C0). Check for solder bridges.
5. Solder the through-hole connectors (J1 screw terminal, J2, J3, J4, J5) by hand using a soldering iron.
6. Clean the board with isopropyl alcohol to remove flux residue.
7. Perform a visual inspection and continuity check with a multimeter before applying power.
8. The PS_ENABLE, SCK_PULLUP, and SDA_PULLUP jumpers are shorted by default in the PCB design, enabling power and I2C pull-ups without manual configuration. These can be disabled by cutting the corresponding PCB traces if needed.

5.4 Firmware flashing

1. Connect an ST-Link V2 programmer to the TC2030 SWD footprint (U6) using a Tag-Connect cable (pins: SWDIO, SWCLK, GND, 3.3 V).
2. Open the project in STM32CubeIDE (available free from STMicroelectronics).
3. Build the project and flash via the Run or Debug menu.
4. Alternatively, use STM32CubeProgrammer with the pre-compiled firmware image (`firmware_Image.elf`) available in the Zenodo repository at <https://doi.org/10.5281/zenodo.20087909>.
5. Upon successful flashing, the device is detectable on the I2C bus at default address 0x20 (7-bit).

5.5 Address configuration

The default I2C address is 0x20 (7-bit). To change it, send `SET_I2C_ADDR` followed by the desired address over UART or I2C; the value is saved to flash and takes effect after a power cycle. Use `READ_I2C_ADDR` to query the current address. No programming tool or firmware reflash is required.

6. Operation instructions

6.1 Electrical connections

Connect the hardware as follows before powering the system:

1. Connect the DC motor and encoder to J4 (6-pin header) according to the pinout in Table 1.
2. Connect the primary power supply (5–45 V DC) to J1 (screw terminal, 3.5 mm pitch). Verify polarity before applying power. Alternatively, motor supply voltage can be provided via the motor power supply pin on J3.
3. Connect the host device via I2C using the Qwiic connectors J2 or J5, or via UART using J3 (pins: 3.3 V, GND, RX, TX).

Table 1: J4 connector pinout (6-pin male header, 2.54 mm pitch).

Pin	Signal	Description
1	M1	Motor terminal 1
2	M2	Motor terminal 2
3	GND	Ground
4	3.3 V	Encoder power supply
5	S2	Encoder channel B
6	S1	Encoder channel A

Safety notice: Do not connect or disconnect the motor while the board is energized. Ensure the supply voltage does not exceed 45 V under any transient condition.

6.2 ASCII command protocol

Commands are sent as plain ASCII strings terminated with a newline (`\n`) character over UART (115200 baud, 8N1) or I2C at 100 kHz. All position values are in encoder counts; velocity in counts/s; acceleration in counts/s². The complete command set is listed in Table 2.

PID gain scaling: All PID gain values commanded via `SET_P`, `SET_I`, and `SET_D` are divided by a factor of 10 internally before being applied to the control output. For example, sending `SET_P 1000` results in an effective proportional gain of $K_p = 100$. Users should account for this factor when tuning.

Table 2: ASCII command set. All commands are terminated with \n.

Command	Description	Example
SETPOS <n>	Set target position in encoder counts; engages PID mode	SETPOS 1000
MOVETO <n>	Move to position using trapezoidal trajectory planner	MOVETO 5000
FLOAT	Set H-bridge to floating (coast) state; disables drive	FLOAT
BRAKE	Set H-bridge to active brake state	BRAKE
GETPOS	Query current encoder position (counts)	GETPOS
GETSPD	Query current speed (encoder ticks per interval)	GETSPD
SET_P <v>	Set PID proportional gain K_p	SET_P 1000
SET_I <v>	Set PID integral gain K_i	SET_I 10
SET_D <v>	Set PID derivative gain K_d	SET_D 5
GET_PID	Query current K_p , K_i , K_d values	GET_PID
SETMAX_VEL <v>	Set maximum trajectory velocity (counts/s)	SETMAX_VEL 20.0
SETMAX_ACC <v>	Set maximum trajectory acceleration (counts/s ²)	SETMAX_ACC 0.05
GET_ADC	Read IPROPI sense resistor output as a qualitative load indicator (SMA-filtered ADC counts; see Section 7.4)	GET_ADC
LOG <ms>	Enable periodic telemetry every ms milliseconds (min. 20 ms). Format: pos <n>, amps <i>, tps <v>	LOG 50
LOG 0	Disable telemetry logging	LOG 0
STATUS	Query current drive mode (FLOAT / BRAKE / PID / TRAPEZOID)	STATUS
CONFIRM <0/1>	Enable (1) or disable (0) verbose confirmation messages	CONFIRM 1
SETSPD <0-100>	Set motor speed as a percentage (open-loop; no PID)	SETSPD 50
SET_I2C_ADDR <v>	Set I2C address (saved to flash; takes effect on next power cycle)	SET_I2C_ADDR 33
READ_I2C_ADDR	Query current I2C address stored in flash	READ_I2C_ADDR

Note on SETSPD: The SETSPD command sets the motor drive PWM duty cycle directly as a percentage (0–100%) in open-loop mode — it does not engage the PID controller or trajectory planner. For closed-loop position control, use SETPOS (direct PID) or MOVETO (trapezoidal trajectory planner).

6.3 Daisy-chaining multiple units

To control multiple motors from a single I2C master:

1. Assign each board a unique I2C address using SET_I2C_ADDR over UART or I2C, then power-cycle to apply.
2. Connect the Qwiic output of one board to the Qwiic input of the next in the chain.
3. Address each board individually in the host I2C write/read transactions.
4. Up to 112 units are supported on a single bus using standard 7-bit I2C addressing.

Figure 2 shows two units daisy-chained via Qwiic connectors with an Arduino as the I2C host.

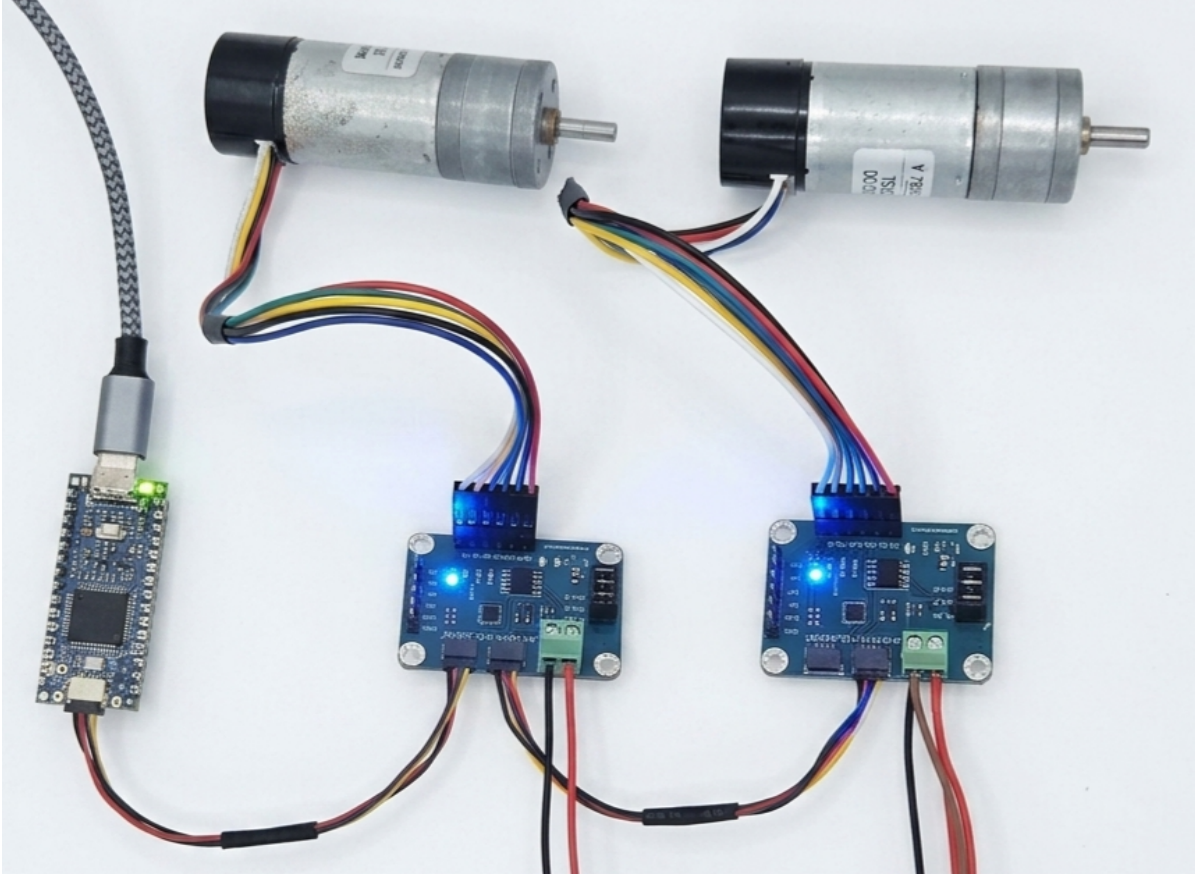


Figure 2: Two Closed-Loop DC Motor Controller boards daisy-chained via Qwiic connectors, with DC motors connected and an Arduino as the I2C host controller.

7. Validation and characterization

The following tests were performed on a fully assembled unit to validate correct operation and characterise performance. All tests were conducted at a supply voltage of 12 V using a TS-25GAH370-20 geared DC motor (12 V, 300 RPM, 20:1 gearbox, 11 PPR encoder). Validation across the full 5–45 V supply range and with multiple motor types is beyond the scope of this work. Users deploying the driver with different motors or supply voltages should retune PID gains empirically using the procedure in Section 7.1.

7.1 Step response

A step position command was issued via SETPOS (bypassing the trapezoidal trajectory planner) with proportional-only control ($K_I = K_D = 0$). The position response was recorded at 250 ms telemetry intervals. Figure 3 shows the measured position versus time.

ANGULAR POSITION & TARGET

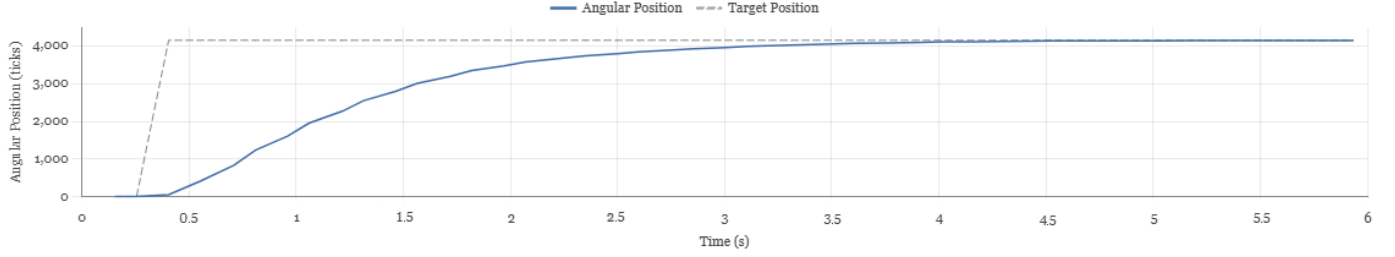


Figure 3: Step response of the closed-loop position controller using SETPOS with proportional-only control. The motor shaft moves from 0 to 4000 encoder counts (approximately 1636° , or 4.5 output shaft revolutions) and stabilises without oscillation. PID gains: $K_P = 1000$ (commanded; effective $K_P = 100$), $K_I = 0$, $K_D = 0$. Supply voltage: 12 V. Test motor: TS-25GAH370-20 (12 V, 300 RPM, 20:1 gearbox, 11 PPR encoder).

Key metrics from the step response:

- Rise time (10% to 90% of target): approximately 2000 ms
- Settling time (within ± 80 counts, $\approx \pm 33^\circ$): approximately 3500 ms
- Overshoot: 0% (overdamped response)
- Steady-state position error: ≈ 0 encoder counts ($< 0.41^\circ$)
- Step size: 4000 counts $\approx 1636^\circ$ (4.5 output shaft revolutions) at $0.41^\circ/\text{count}$

PID tuning guidance. All validation tests used proportional-only control ($K_I = K_D = 0$), which was found sufficient for the low-inertia, low-friction motor used in testing. For higher-inertia loads or applications requiring tighter steady-state accuracy, integral and derivative terms can be enabled at runtime via SET_I and SET_D without reflashing firmware. A recommended starting procedure: (1) set $K_I = K_D = 0$ and increase K_P until the response is fast but shows mild overshoot; (2) add K_D incrementally to damp the overshoot (typical starting values: $K_D \approx 0.1 \times K_P$); (3) add a small K_I only if a steady-state error remains (typical: $K_I \approx 0.01 \times K_P$).

7.2 Trapezoidal trajectory tracking

The trapezoidal trajectory planner was evaluated by commanding the motor to sweep between 0 and 3000 encoder counts using MOVETO. Figure 4 shows position and velocity for a representative single cycle under no load, and Figure 5 shows five consecutive cycles to demonstrate repeatability.

Settling errors at the end of each half-cycle were 46 counts ($\approx 18.9^\circ$) for the move to 3000 and 48 counts ($\approx 19.7^\circ$) for the return to 0. Across five consecutive no-load cycles, the final position at target 3000 had a standard deviation of 1.6 counts (0.67°) and a total range of 4 counts (1.6°), demonstrating highly consistent cycle-to-cycle repeatability.

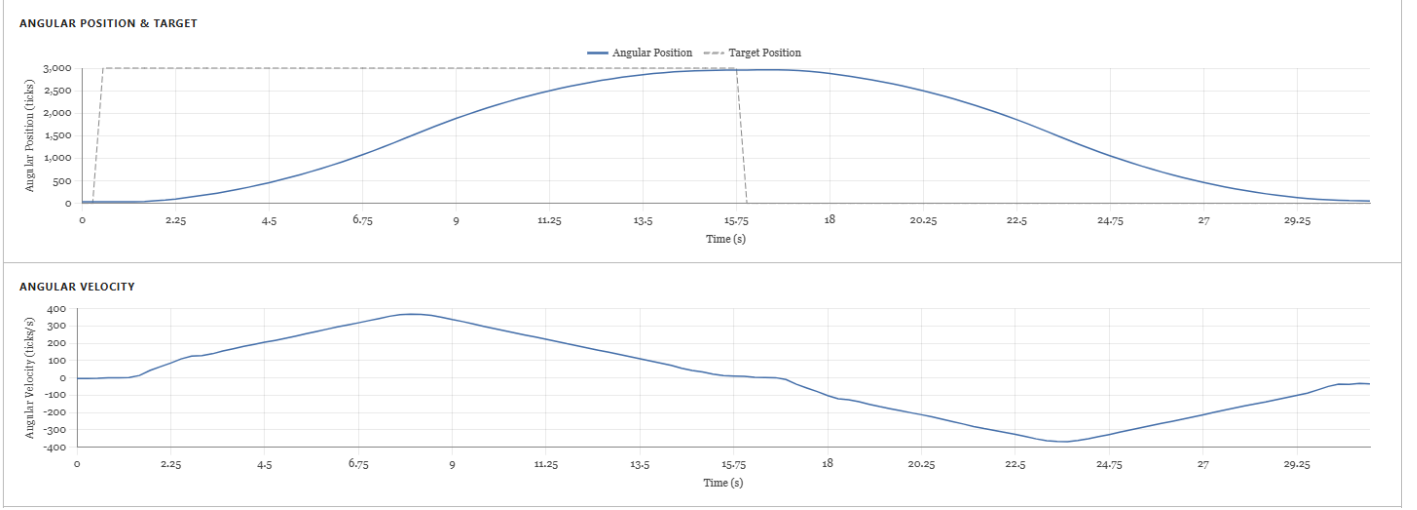


Figure 4: Single-cycle trapezoidal trajectory (no load). Top: measured encoder position (counts). Bottom: angular velocity (counts s^{-1}). The motor sweeps from 0 to 3000 encoder counts ($\approx 1227^\circ$) and returns to 0. Supply voltage: 12 V.

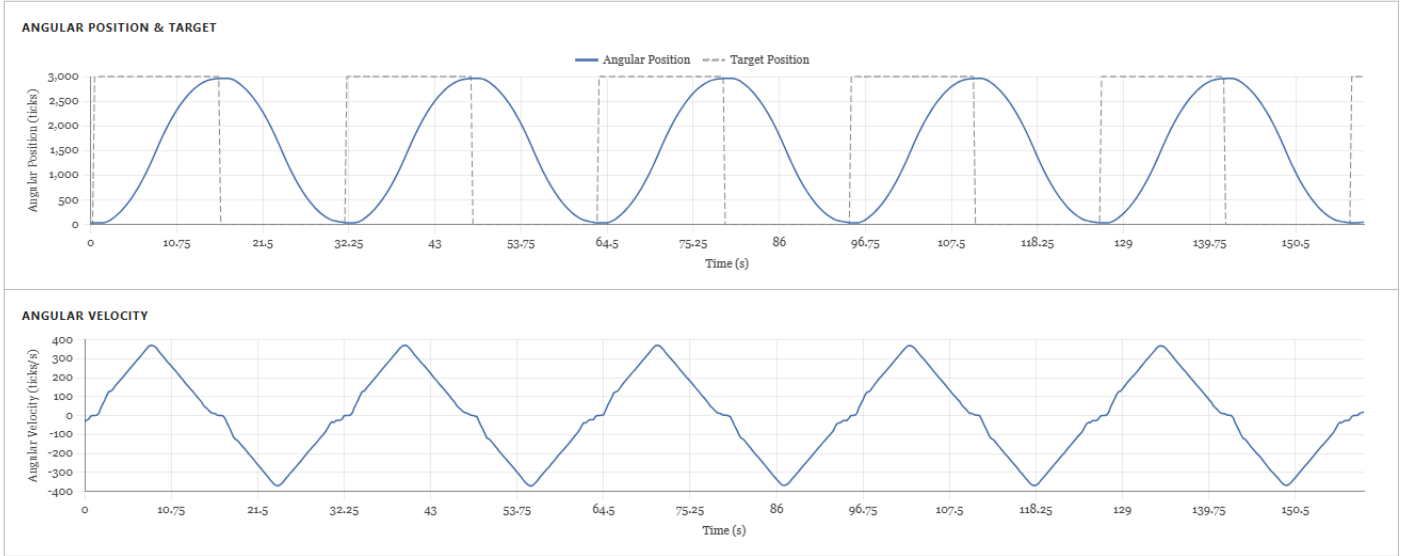


Figure 5: Five consecutive trapezoidal trajectory cycles (no load), demonstrating cycle-to-cycle consistency. Top: measured encoder position (counts). Bottom: angular velocity (counts s^{-1}). Peak velocity of $\approx 375 \text{ counts s}^{-1}$ corresponds to $\approx 25.6 \text{ RPM}$ at the output shaft.

7.3 Trajectory tracking under load

To evaluate the effect of mechanical load on tracking performance, two additional single-cycle trajectory tests were conducted with a second geared DC motor coupled shaft-to-shaft to the test motor. Load level was varied by controlling the terminal condition of the coupled motor. Figure 6 shows results with the coupled motor terminals open circuit (light load), and Figure 7 shows results with the coupled motor terminals short-circuited (heavy load). All tests used identical trajectory parameters and proportional-only control ($K_P = 2000$, effective $K_P = 200$, $K_I = K_D = 0$).

Settling errors are summarised in Table 3. Under light load, final settling errors were 96 counts ($\approx 39.4^\circ$) and 94 counts ($\approx 38.5^\circ$). Under heavy load, settling errors were 87 counts ($\approx 35.7^\circ$) and 89 counts ($\approx 36.5^\circ$), comparable to the light-load case. The similarity between conditions indicates that the dominant source of settling error under load is static gearbox friction rather than back-EMF braking torque. These settling errors are expected under proportional-only control in the presence of a constant disturbance torque [8,9] and are suitable for the primary target applications of this hardware. An integral term can be added at runtime via `SET_I` for applications requiring tighter steady-state accuracy under load.

Table 3: Final settling error for each load condition and move direction. Values in encoder counts and equivalent degrees ($0.41^\circ/\text{count}$).

Condition	Move to 3000		Return to 0	
	counts	degrees	counts	degrees
No load	46	18.9°	48	19.7°
Light load	96	39.4°	94	38.5°
Heavy load	87	35.7°	89	36.5°

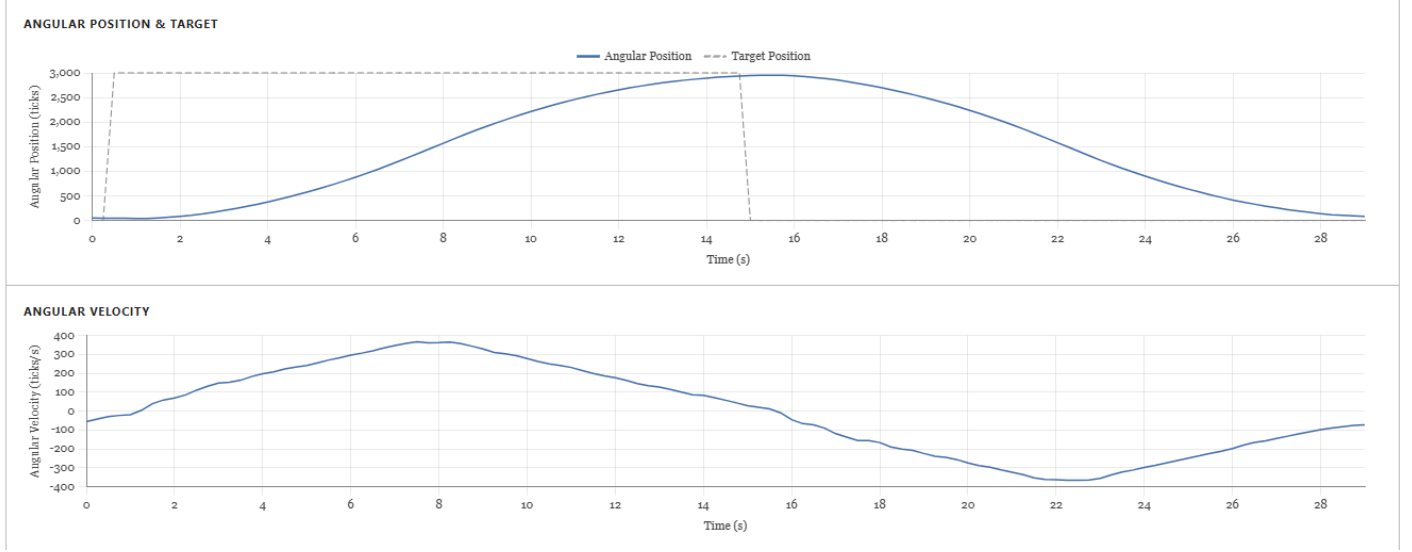


Figure 6: Single-cycle trapezoidal trajectory under light load. Coupled motor terminals open circuit. Commanded $K_P = 2000$ (effective $K_P = 200$), $K_I = K_D = 0$. Supply voltage: 12 V.

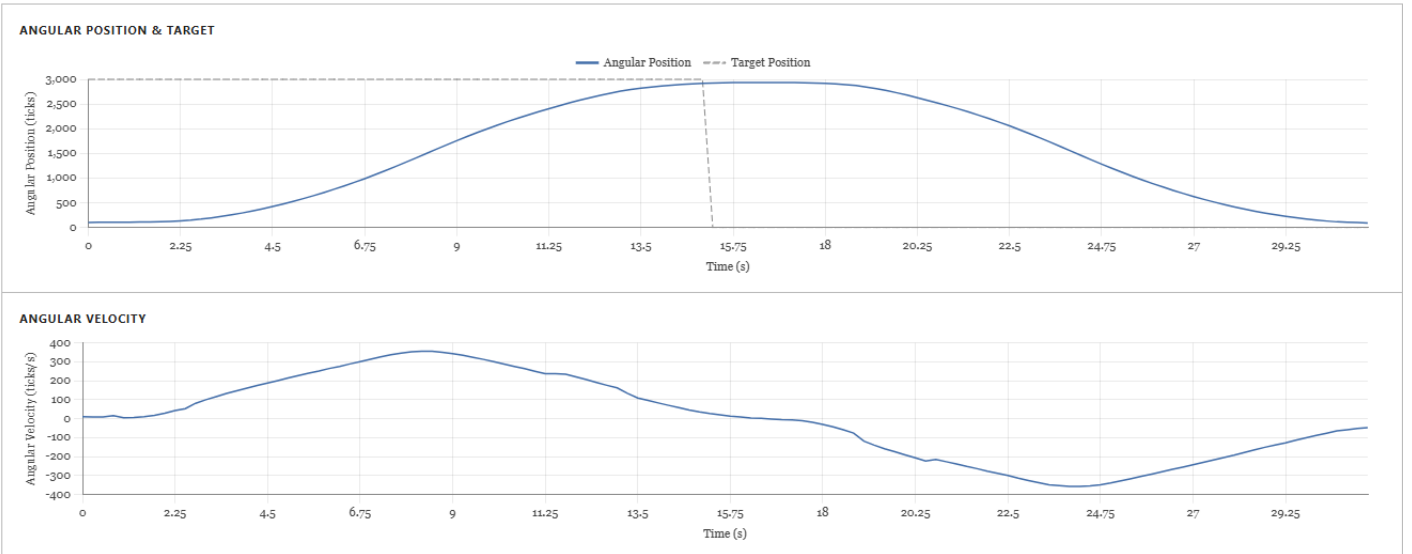


Figure 7: Single-cycle trapezoidal trajectory under heavy load. Coupled motor terminals short-circuited to maximise regenerative braking torque. Conditions otherwise identical to Figure 6.

7.4 Current draw characterization

Motor current is sensed via the IPROPI output of the DRV8251A, which sources a current proportional to the motor drive current through the 1.5 k Ω sense resistor R1, producing a voltage read by the STM32 12-bit ADC on channel 8. During validation tests, raw ADC readings exhibited significant noise attributed to PWM switching transients and digital noise coupling into the sense line. Two software mitigations were applied: (1) a 20-sample simple moving average filter in firmware (`filter_sma()` in `main_original.c`); and (2) the ADC sampling time can be increased to `ADC_SAMPLETIME_160CYCLES_5` to further improve noise rejection.

For applications requiring accurate quantitative current measurement, an RC low-pass filter is recommended on the hardware sense line: a 1 k Ω resistor in series with a 100 nF capacitor to ground gives a cutoff frequency of approximately 1.6 kHz, attenuating switching noise while preserving the current envelope. This RC filter can be added in a future PCB revision with no changes to the existing schematic topology. The current sense output in its present form is suitable for qualitative load monitoring (e.g., detecting motor stall) but is not recommended for quantitative torque estimation without the hardware filter described above.

7.5 Capabilities and limitations

- Supply voltage range: 5–45 V per DRV8251A specification; validated at 12 V only
- Peak output current: 4.1 A (component-level rating of DRV8251ADDAR)
- *Thermal note:* Continuous current is practically limited to approximately 1.6 A RMS on the standard 2-layer, 1 oz copper PCB without additional thermal management. At 4.1 A, estimated power dissipation is ≈ 7.6 W and junction temperature rise would exceed the 150 $^{\circ}\text{C}$ absolute maximum. Sustained high-current operation requires enlarged copper pours or active cooling.
- PCB dimensions: 30 mm $\times$ 40 mm; mass: 8 g (fully assembled)
- Microcontroller: STM32C011F6U6TR, ARM Cortex-M0+, 48 MHz (HSI)
- PID loop update rate: estimated 100–200 kHz under idle conditions; not directly measured
- Encoder type: incremental quadrature (magnetic or optical); 64-bit position counter
- Test motor encoder resolution: 880 counts/revolution at output shaft; 0.41 $^{\circ}$ /count
- PWM resolution: 12-bit effective (TIM1, period = 4095 counts)
- PWM frequency: approximately 11.7 kHz
- Current sensing: 12-bit ADC, IPROPI method; qualitative only in current revision (see Section 7.4)
- Communication: UART at 115200 baud (8N1); I2C at 100 kHz, default address 0x20
- Maximum I2C daisy-chain: 112 units (standard 7-bit addressing)
- Trajectory planner: trapezoidal velocity profile with configurable velocity and acceleration limits
- Drive modes: PID position control, trapezoidal trajectory, float (coast), brake
- Operating temperature: 0 $^{\circ}\text{C}$ to 85 $^{\circ}\text{C}$ (limited by STM32C011F6U6TR and DRV8251ADDAR ratings)
- *Limitation:* Brushed DC motors only; brushless motors not supported
- *Limitation:* Position is relative; absolute position not retained across power cycles. A homing routine is required if absolute positioning is needed.
- *Limitation:* I2C address changes take effect only after a power cycle
- *Limitation:* PID loop execution rate depends on main loop timing rather than a dedicated hardware interrupt; rate varies with communication activity

Ethics statements

This work involves no human subjects and no animal experiments.

CRedit author statement

[Removed for double-blind review]

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Claude (Anthropic) to assist with manuscript drafting, editing, and technical writing. After using this tool, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Acknowledgements

This work was supported in part by a grant from the Tufts Derby Entrepreneurship Center, awarded following a verbal presentation of this project to a review committee.

Declaration of interest

Declarations of interest: none.

References

- [1] ODrive Robotics. ODrive S1 Motor Controller. ODrive Robotics Inc., El Sobrante, CA, USA. Available at: <https://odriverobotics.com>. Accessed April 2025.
- [2] ODrive Robotics. ODrive S1 Datasheet. ODrive Documentation v0.6.11. Available at: <https://docs.odriverobotics.com/v/latest/hardware/s1-datasheet.html>. Accessed April 2025.
- [3] Pololu Corporation. Jrk G2 21v3 USB Motor Controller with Feedback. Pololu product #3142. Available at: <https://www.pololu.com/product/3142>. Accessed April 2025.
- [4] STMicroelectronics. STM32C011F6 Datasheet — Arm Cortex-M0+ 32-bit MCU. Doc. DS14031. Available at: <https://www.st.com/en/microcontrollers-microprocessors/stm32c011f6.html>. Accessed April 2025.
- [5] Texas Instruments. DRV8251A 5.5-V to 45-V, 4.1-A H-Bridge Motor Driver Datasheet. Doc. SLVSFF4. Available at: <https://www.ti.com/product/DRV8251A>. Accessed April 2025.
- [6] SparkFun Electronics. Qwiic Connect System. Available at: <https://www.sparkfun.com/qwiic>. Accessed April 2025.
- [7] NXP Semiconductors. I²C-bus specification and user manual. Rev. 7.0. Doc. UM10204. 2021. Available at: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>. Accessed April 2025.
- [8] Åström K.J., Hägglund T. *PID Controllers: Theory, Design, and Tuning*. 2nd ed. Research Triangle Park, NC: Instrument Society of America; 1995.
- [9] Franklin G.F., Powell J.D., Emami-Naeini A. *Feedback Control of Dynamic Systems*. 8th ed. Hoboken, NJ: Pearson; 2019.
- [10] Biagiotti L., Melchiorri C. *Trajectory Planning for Automatic Machines and Robots*. Berlin: Springer; 2008.
- [11] Grout V. Open-source hardware in research: a review of current practice. *HardwareX*. 2017;1:1–8. <https://doi.org/10.1016/j.ohx.2016.09.001>.
- [12] Allain R., Zettler M., Horne B., Perreault S. SimpleFOC: An open-source field-oriented control library for brushless DC motors. *HardwareX*. 2022;11:e00296. <https://doi.org/10.1016/j.ohx.2022.e00296>.